

# Estado del Arte

Jesús Alejandro Rivas Reinoso

*Carrera de Computación, Universidad  
Técnica Particular de Loja (UTPL)*

**Abstract**—Obtener datos es solo el primer paso. El verdadero desafío está en transformarlos: limpiarlos, estructurarlos, hacerlos coherentes entre sí y garantizar que ese proceso pueda repetirse sin depender de una persona que recuerde cómo hacerlo. Este artículo revisa el estado del arte en ingeniería de datos con foco en las etapas de transformación, modelado relacional y organización de pipelines reproducibles. La revisión se apoya en trabajos empíricos y arquitecturas documentadas que abordan el problema desde distintos ángulos: desde encuestas sobre herramientas de uso real hasta casos de aplicación en entornos institucionales. Los hallazgos se contrastan con las decisiones de diseño adoptadas en un proyecto concreto de desarrollo de pipeline en curso, el Pipeline de Datos del Macroentorno Ecuatoriano, construido dentro de la DGTITD de la Universidad Técnica Particular de Loja, que enfrenta exactamente el tipo de retos que la literatura describe: fuentes heterogéneas, procesos manuales no documentados y necesidad de un modelo que sea mantenible en el tiempo.

## I. Introducción

Recopilar datos no es suficiente. El valor analítico de cualquier fuente de información depende de cuánto esfuerzo se invierte en limpiarla, estructurarla y hacerla coherente con otras fuentes. Majid et al. [3] estiman que las actividades de adquisición y preparación de datos consumen hasta el 80% del tiempo en proyectos analíticos reales, y Biswas et al. [7] confirman que estos pasos están presentes en prácticamente todos los flujos de trabajo, independientemente del dominio. A pesar de eso, en muchas organizaciones estos procesos siguen siendo manuales, no documentados y dependientes de personas específicas, lo que los hace frágiles ante cualquier cambio en las fuentes o en el equipo.

Ese es exactamente el escenario que motivó este trabajo. La DGTITD de la UTPL mantiene un tablero de indicadores institucionales que integra datos de nueve fuentes públicas ecuatorianas, pero su actualización depende de pasos manuales no documentados: cuando cambia un archivo fuente, el conocimiento de qué hacer vive en personas, no en código. Construir el pipeline que formalice ese proceso es el objetivo del prácticum en el que se enmarca este artículo. El problema no es acceder a los datos, sino transformarlos de forma confiable: cada fuente tiene su propia estructura, sus propios formatos de fecha, sus propios valores nulos y sus propias convenciones de columnas.

Este artículo presenta la revisión de literatura que sustenta las decisiones de diseño del pipeline, con foco en tres preguntas: (1) ¿Qué arquitecturas organizan mejor el flujo de transformación desde datos crudos hasta indicadores listos para visualizar? (2) ¿Cómo estructurar el modelo relacional de la capa Silver para garantizar calidad y facilidad de mantenimiento? y (3) ¿Qué enfoques documentados en la literatura son aplicables a pipelines de alcance institucional con recursos tecnológicos acotados? El desarrollo actual, que al cierre de la semana 2 tiene procesadas las fuentes del BCE

con Python y pandas, sirve como caso de contraste concreto a lo largo de la revisión.

## II. MARCO CONCEPTUAL

### A. Pipelines de Datos

Un pipeline de datos es una serie de pasos de procesamiento automatizados diseñados para mover, transformar y entregar información desde su origen hasta un destino adecuado para análisis. Según Majid et al. [3], el propósito central es transformar datos crudos en una forma estructurada que sirva como entrada a tareas posteriores; estas actividades consumen hasta el 80% del tiempo de trabajo en un proyecto de IA. Biswas et al. [7] encontraron que la adquisición, preparación y almacenamiento de datos están presentes en prácticamente todos los flujos de trabajo reales, independientemente del dominio de aplicación.

### B. Arquitecturas ETL y ELT

Históricamente, la ingeniería de datos operó bajo el paradigma Extraer–Transformar–Cargar (ETL): los datos se extraen, se transforman en un servidor intermedio y se cargan en la base destino. Sin embargo, su dependencia de una capa de transformación separada introduce cuellos de botella al manejar datos de alta velocidad o no estructurados [4]. La aparición de almacenes de datos en la nube habilitó el paradigma ELT (Extraer–Cargar–Transformar), donde los datos crudos se cargan directamente y las transformaciones se ejecutan después, aprovechando la potencia nativa del almacén. Armbrust et al. [4] demuestran que la arquitectura Lakehouse permite tanto la retención flexible de datos crudos como transformaciones de alto rendimiento sin servidor ETL dedicado. En proyectos de alcance institucional con recursos tecnológicos acotados, como el pipeline de la DGTITD-UTPL, el paradigma ETL sigue siendo pertinente: la transformación ocurre antes de la carga porque el destino es una base SQLite ligera, no un almacén en la nube. Esta elección es coherente con lo que Majid et al. [3] describen como pipelines orientados a entornos con infraestructura limitada, donde la simplicidad operativa tiene más valor que la escalabilidad horizontal.

### C. Orquestación de Datos

La orquestación coordina automáticamente múltiples tareas interdependientes dentro de un pipeline. Apache Airflow implementa el paradigma Workflows as Code mediante grafos acíclicos dirigidos (DAGs) en Python, y es utilizado por organizaciones como Google, Adobe y PayPal para gestionar flujos de datos complejos [5]. Mir et al. [5] documentan, sin embargo, que la configuración y el mantenimiento de Airflow presentan una curva de aprendizaje considerable, lo que ha impulsado el desarrollo de alternativas más ligeras como Prefect. En pipelines de etapa temprana o alcance acotado, como el que se desarrolla en la DGTITD-UTPL, la orquestación formal suele implementarse mediante scripts secuenciales en Python antes de migrar a herramientas dedicadas: una práctica que la literatura reconoce como punto de partida válido y frecuente en equipos pequeños [3].

## III. TRABAJOS RELACIONADOS

Rana et al. [1] ofrecen una visión de conjunto sobre el estado actual de la ingeniería de datos, subrayando que la disciplina ha madurado más en sus herramientas que en sus prácticas de documentación y reproducibilidad, una brecha

que proyectos como el pipeline de la DGTITD-UTPL buscan cerrar desde el ámbito institucional. Majid et al. [3] categorizan las soluciones disponibles en cuatro grupos: pipelines ETL/ELT, herramientas de ingesta y transformación, plataformas de orquestación y pipelines de aprendizaje automático, clasificación que resulta útil para situar el alcance del proyecto en desarrollo dentro del espectro más amplio de la disciplina. Heck [2] mapea las actividades de ingeniería de datos a lo largo del ciclo de vida de sistemas de IA, identificando la preparación y transformación como las actividades más estudiadas, con escasez relativa en gobernanza y observabilidad.

Armbrust et al. [4] introducen la arquitectura Lakehouse, que combina la flexibilidad de los data lakes con el rendimiento de los almacenes tradicionales. Mir et al. [5] realizan un estudio empírico sobre los desafíos al implementar Workflows as Code en Airflow, analizando más de 9.000 publicaciones en Stack Overflow. Biswas et al. [7] presentan un estudio empírico a gran escala que confirma la universalidad de los patrones de adquisición y preparación de datos en pipelines reales.

#### IV. CASOS DE ESTUDIO Y APLICACIONES

##### A. Entornos de Smart Campus Universitario

Hernando-Cánovas et al. [8] desarrollaron un sistema predictivo para estimar la ocupación física en una biblioteca universitaria usando datos anónimos de conexiones WiFi. El pipeline extrae continuamente registros de telemetría de puntos de acceso inalámbricos, procesa estas variables y alimenta modelos de aprendizaje automático para producir estimaciones de ocupación sin requerir cámaras ni datos de identificación personal. Los autores concluyen que esta solución es robusta, rentable y preserva la privacidad [8], ilustrando cómo un pipeline bien estructurado transforma telemetría de bajo nivel en inteligencia operativa de alto valor.

##### B. Industria Financiera

Ahonen [10] desarrolló una arquitectura para integrar datos desde sistemas bancarios heredados hacia plataformas modernas en la nube, utilizando Apache Spark Structured Streaming con Azure Event Hubs para capturar eventos continuos de cambio y mantener un perfil de cliente unificado. La latencia de extremo a extremo reportada de entre 1,2 y 1,4 segundos demuestra la viabilidad de Structured Streaming para pipelines empresariales en tiempo real; Ahonen concluye que Databricks es un enfoque viable y de alto rendimiento para estos escenarios [10].

#### V. HERRAMIENTAS Y TECNOLOGÍAS

La Tabla I resume las principales herramientas identificadas en la literatura revisada, alineadas con la taxonomía de Majid et al. [3].

TABLE I. TABLA I

| TABLE II. PRINCIPALES HERRAMIENTAS DE INGENIERÍA DE DATOS |                            |                                                                         |
|-----------------------------------------------------------|----------------------------|-------------------------------------------------------------------------|
| Herramienta                                               | Función principal          | Característica destacada                                                |
| Apache Spark                                              | Procesamiento distribuido  | Motor en memoria para analítica masiva y streaming de baja latencia [9] |
| Python / Scala                                            | Lógica y limpieza de datos | Lenguajes versátiles para datasets complejos e integración con ML [3]   |

|                          |                                |                                                                        |
|--------------------------|--------------------------------|------------------------------------------------------------------------|
| Databricks               | Plataforma analítica unificada | Entorno cloud que optimiza Spark para producción empresarial [9]       |
| Azure Event Hubs         | Ingesta en tiempo real         | Transmisión de eventos de alta capacidad desde sistemas heredados [10] |
| dbt                      | Transformación ELT             | Transforma datos con SQL versionado dentro del almacén [7]             |
| Apache Airflow / Prefect | Orquestación de flujos         | Coordinación automatizada, reintentos y monitoreo de dependencias [5]  |
| Snowflake                | Almacén cloud                  | Repositorio columnar que desacopla almacenamiento y cómputo [4]        |
| Power BI / Tableau       | Visualización e BI             | Convierte datos procesados en indicadores interactivos [1]             |

#### VI. TENDENCIAS Y FUTURO

**Datos listos para IA y pipelines agénticos:** El cambio más significativo es la transición desde pipelines que mueven tablas hacia pipelines que preparan entradas contextuales para agentes de IA autónomos. Heck [2] muestra que los pasos de preprocesamiento e ingeniería de características se han convertido en la restricción de diseño primaria, requiriendo que los pipelines generen metadatos enriquecidos y representaciones vectoriales de forma nativa.

**Observabilidad de datos y gobernanza como código:** Ehrlinger y Rusz [11] identifican un desplazamiento hacia la detección automática de anomalías y el seguimiento de linaje que opera continuamente dentro del stack del pipeline. Este enfoque permite detectar deriva de esquema o registros corruptos antes de que afecten paneles de control o modelos de aprendizaje automático.

**Edge Data Engineering:** Hamid et al. [14] documentan cómo el procesamiento en el borde reduce la latencia al habilitar transformaciones cerca de la fuente. Para la ingeniería de datos, implica diseñar pipelines sobre topologías distribuidas que filtran y agregan datos de sensores de alta frecuencia en el borde antes de transmitir registros consolidados a la nube.

#### VII. CONCLUSIONES

La transformación es el núcleo del pipeline: La literatura revisada confirma que la preparación y limpieza de datos no es una tarea secundaria sino la etapa más costosa y crítica de cualquier proyecto analítico. Esta realidad se refleja directamente en el desarrollo del pipeline de la DGTITD-UTPL, donde cada fuente ha requerido decisiones específicas de transformación antes de poder integrarse al modelo relacional.

La arquitectura define lo que es mantenible: La separación entre capas Bronze, Silver y Gold, respaldada por trabajos como los de Armbrust et al. [4] y Majid et al. [3], no es una decisión estética sino funcional. En proyectos con fuentes de actualización periódica, tener capas bien delimitadas es lo que permite modificar una transformación sin romper el resto del flujo.

Los casos de estudio validan el enfoque: Los trabajos de Hernando-Cánovas et al. [8] y Ahonen [10] muestran que el factor diferenciador en proyectos analíticos exitosos no es la sofisticación del algoritmo sino la solidez del pipeline que lo

alimenta. Esa lección es directamente aplicable al contexto institucional en el que se desarrolla este trabajo.

Reproducibilidad como objetivo de diseño: Documentar las decisiones de transformación y formalizar el pipeline en código no es un entregable adicional: es lo que convierte un proceso frágil en uno sostenible. La literatura y el proyecto coinciden en que ese es el punto de partida real de cualquier sistema analítico institucional.

#### REFERENCIAS

- [1] [1] D. Rana, A. Bhatt, R. B. Dubey y S. K. Bhatt, “Data Engineering: An Overview from a Future Perspective,” en *Proc. 2023 IEEE Int. Conf. on Contemporary Computing and Communications (InC4)*, IEEE, 2023. DOI: 10.1109/InC4.2023.10398128
- [2] [2] P. Heck, “What About the Data? A Mapping Study on Data Engineering for AI Systems,” en *Proc. CAIN 2024*, ACM, Lisboa, abr. 2024. DOI: 10.1145/3644815.3644954
- [3] [3] A. Majid, S. Laue, K. Böhm y N. Piatkowski, “A Survey of Pipeline Tools for Data Engineering,” *arXiv preprint arXiv:2406.08335*, 2024.
- [4] [4] M. Armbrust et al., “Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics,” en *Proc. 11th CIDR 2021*, 2021.
- [5] [5] H. Mir, D. Rajbahadur y Y. Tian, “An Empirical Study of Developers’ Challenges in Implementing Workflows as Code: Apache Airflow,” *J. Systems and Software*, vol. 217, 2024. DOI: 10.1016/j.jss.2024.112029
- [6] [7] S. Biswas, M. Wardat y H. Rajan, “The Art and Practice of Data Science Pipelines,” en *Proc. ICSE 2022*, pp. 2091–2103. DOI: 10.1145/3510003.3510057
- [7] [8] L. Hernando-Cánovas et al., “A Machine Learning Approach for Estimating Person Counts Using Anonymous WiFi Data in a University Library,” *Sensors*, vol. 25, n.º 22, p. 7065, 2025. DOI: 10.3390/s25227065
- [8] [9] A. Alzahrani, B. Alharbi y K. Al-Tawil, “Apache Spark in Riot Games: A Case Study on Data Processing,” *IJACSA*, vol. 14, n.º 7, 2023. DOI: 10.14569/IJACSA.2023.0140704
- [9] [10] J. Ahonen, “Real-Time Customer Data Platform using Apache Spark Structured Streaming and Databricks,” *Tesis de Máster*, Tampere University, 2025.
- [10] [11] L. Ehrlinger y W. Ruzs, “A Survey of Data Quality Measurement and Monitoring Tools,” *Frontiers in Big Data*, vol. 5, 2022. DOI: 10.3389/fdata.2022.850611
- [11] [14] A. Hamid et al., “Enhancing IoT Efficiency by Minimizing Latency with Edge Computing,” *SN Computer Science*, Springer, 2025. DOI: 10.1007/s42979-025-04192-x

